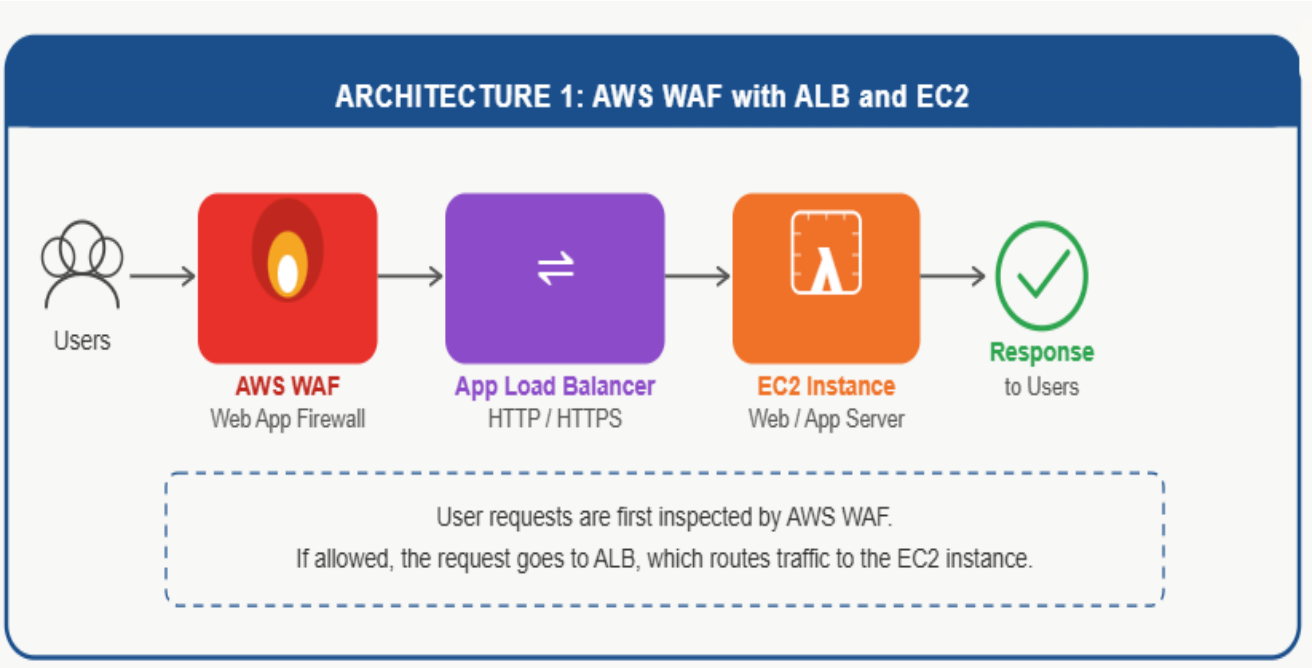
# Challenge 07

- **AWS Cloud Architecture Overview**
- **AWS Infrastructure Design: WAF, ALB & EC2 with Secrets Management**
- **Cloud Architecture Reference Guide**




**Architecture**

**Users**

▼

**AWS WAF**

▼

**Application Load Balancer**

▼

**EC2 Instance**

**Deliverable Summary**

**Service Used: AWS WAF, ALB, EC2**

**Objective: Secure web application from common web attacks.**

**Result: WAF filters malicious requests before they reach the EC2 application through the Load Balancer.**

### 1. Verify EC2 Web Server

SSH into EC2 and run:

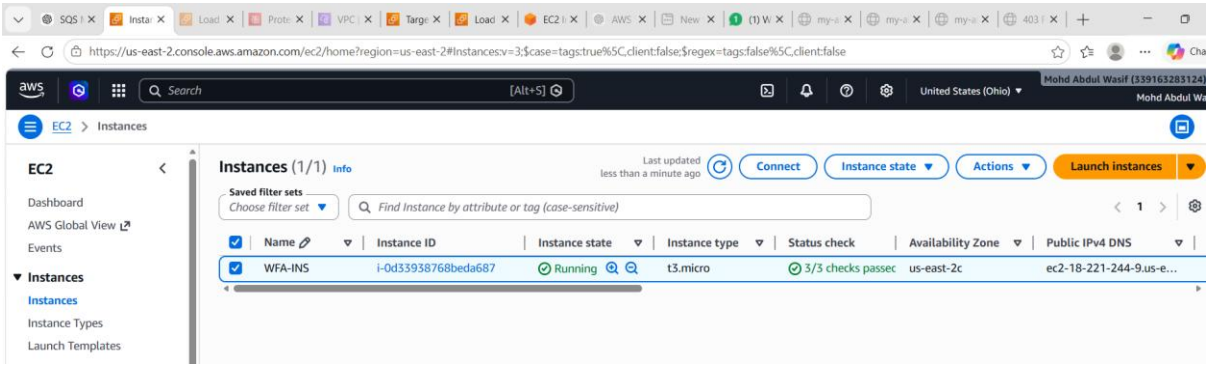
sudo systemctl status httpd

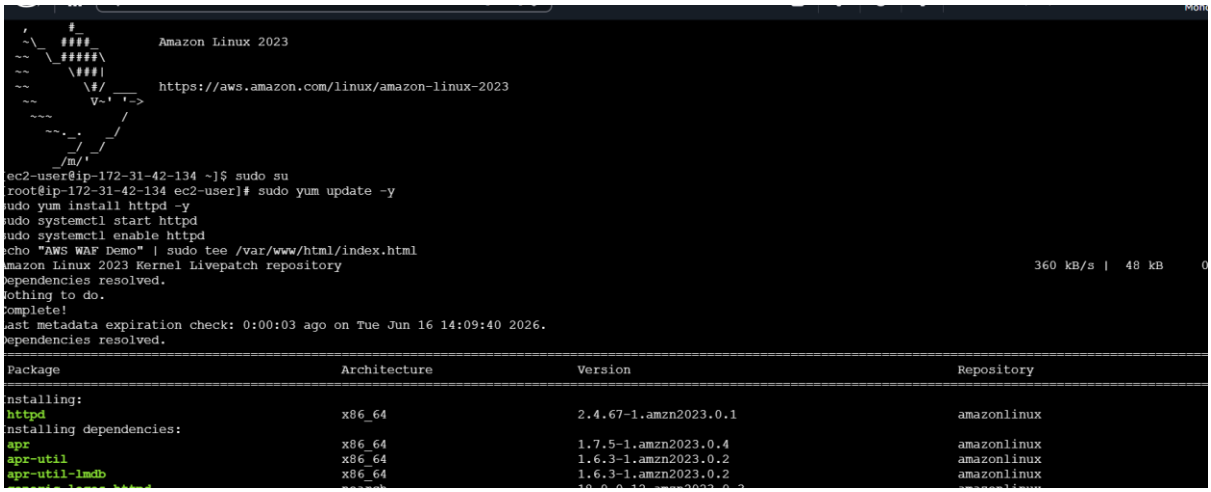
If stopped:

sudo systemctl start httpd
sudo systemctl enable httpd

Test locally:

curl localhost

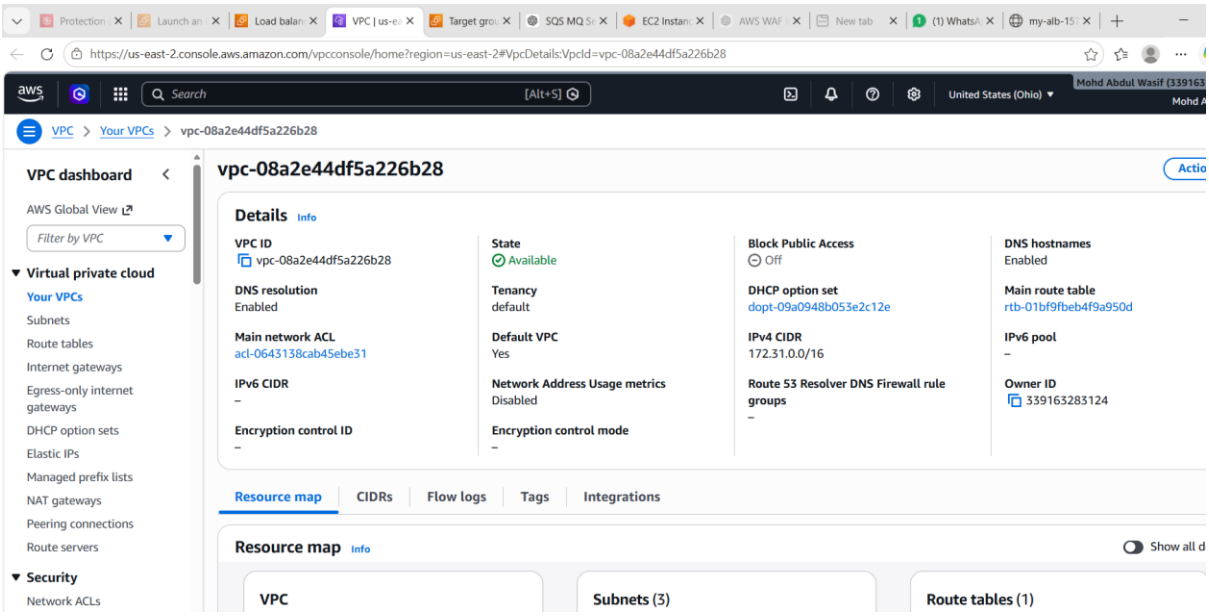


```
Installing weak dependencies:
 apr-util-openssl                x86_64                1.6.3-1.amzn2023.0.2                amazonlinux
 mod_http2                       x86_64                2.0.39-1.amzn2023.0.1               amazonlinux
 mod_lua                         x86_64                2.4.67-1.amzn2023.0.1               amazonlinux

Transaction Summary
================================================================================
Install  13 Packages

Total download size: 2.4 M
Installed size: 7.0 M
Downloading Packages:
(1/13): apr-util-lmdb-1.6.3-1.amzn2023.0.2.x86_64.rpm      358 kB/s |  13 kB     00:00
(2/13): apr-util-1.6.3-1.amzn2023.0.2.x86_64.rpm           2.3 MB/s |  97 kB     00:00
(3/13): apr-1.7.5-1.amzn2023.0.4.x86_64.rpm                2.7 MB/s | 129 kB     00:00
(4/13): apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64.rpm   520 kB/s |  15 kB     00:00
(5/13): generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch.rpm 649 kB/s |  19 kB     00:00
(6/13): httpd-2.4.67-1.amzn2023.0.1.x86_64.rpm             1.6 MB/s |  46 kB     00:00
(7/13): httpd-core-2.4.67-1.amzn2023.0.1.x86_64.rpm         44 MB/s | 1.4 MB     00:00
(8/13): httpd-filesystem-2.4.67-1.amzn2023.0.1.noarch.rpm  381 kB/s |  12 kB     00:00
(9/13): httpd-tools-2.4.67-1.amzn2023.0.1.x86_64.rpm       2.4 MB/s |  80 kB     00:00
(10/13): mailcap-2.1.49-3.amzn2023.0.3.noarch.rpm          1.5 MB/s |  33 kB     00:00
(11/13): libbrotli-1.0.9-4.amzn2023.0.2.x86_64.rpm         9.4 MB/s | 315 kB     00:00
(12/13): mod_http2-2.0.39-1.amzn2023.0.1.x86_64.rpm        5.0 MB/s | 167 kB     00:00
(13/13): mod_lua-2.4.67-1.amzn2023.0.1.x86_64.rpm          2.6 MB/s |  59 kB     00:00
--------------------------------------------------------------------------------
Total                                                       12 MB/s | 2.4 MB     00:00
Running transaction check
```

```
  Verifying        : mod_http2-2.0.39-1.amzn2023.0.1.x86_64
  Verifying        : mod_lua-2.4.67-1.amzn2023.0.1.x86_64

Installed:
  apr-1.7.5-1.amzn2023.0.4.x86_64                apr-util-1.6.3-1.amzn2023.0.2.x86_64                apr-util-lmdb-1.6.3-1.amzn202
  apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64   generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch   httpd-2.4.67-1.amzn2023.0.1.x
  httpd-core-2.4.67-1.amzn2023.0.1.x86_64        httpd-filesystem-2.4.67-1.amzn2023.0.1.noarch       httpd-tools-2.4.67-1.amzn2023
  libbrotli-1.0.9-4.amzn2023.0.2.x86_64          mailcap-2.1.49-3.amzn2023.0.3.noarch                mod_http2-2.0.39-1.amzn2023.0
  mod_lua-2.4.67-1.amzn2023.0.1.x86_64

Complete!
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
AWS WAF Demo
[root@ip-172-31-42-134 ec2-user]# sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
     Active: active (running) since Tue 2026-06-16 14:09:46 UTC; 10min ago
       Docs: man:httpd.service(8)
   Main PID: 3897 (httpd)
     Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec:   0 B/sec"
      Tasks: 177 (limit: 1014)
     Memory: 13.5M
        CPU: 661ms
     CGroup: /system.slice/httpd.service
             ├─3897 /usr/sbin/httpd -DFOREGROUND
             ├─3902 /usr/sbin/httpd -DFOREGROUND
             ├─3906 /usr/sbin/httpd -DFOREGROUND
             ├─3909 /usr/sbin/httpd -DFOREGROUND
             └─4036 /usr/sbin/httpd -DFOREGROUND

Jun 16 14:09:46 ip-172-31-42-134.us-east-2.compute.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Jun 16 14:09:46 ip-172-31-42-134.us-east-2.compute.internal httpd[3897]: Server configured, listening on: port 80
```

```
             ├─3909 /usr/sbin/httpd -DFOREGROUND
             └─4036 /usr/sbin/httpd -DFOREGROUND

Jun 16 14:09:46 ip-172-31-42-134.us-east-2.compute.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Jun 16 14:09:46 ip-172-31-42-134.us-east-2.compute.internal httpd[3897]: Server configured, listening on: port 80
Jun 16 14:09:46 ip-172-31-42-134.us-east-2.compute.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
[root@ip-172-31-42-134 ec2-user]# sudo systemctl start httpd
sudo systemctl enable httpd
[root@ip-172-31-42-134 ec2-user]# curl localhost
AWS WAF Demo
[root@ip-172-31-42-134 ec2-user]#
```




### Check Target Group Health

Go to:

**EC2 → Target Groups → Targets**

Look at the target status.

- Healthy ✅ → Target is fine.
- Unhealthy ❌ → Check the reason shown.

Common reasons:

- Health check path incorrect.
- Port mismatch.
- Security Group blocking traffic.

---

### 3. Verify Security Group

EC2 Security Group should allow:

| Type | Port | Source |
|---|---|---|
| HTTP | 80 | ALB Security Group or 0.0.0.0/0 |

---

### 4. Check Health Check Settings

Target Group → Health Checks

Typical settings:

- Protocol: HTTP
- Port: Traffic Port
- Path: /

If your application is Tomcat:

- Path: /
  or
- Path: /appname

---

**5. Test Directly on EC2**

Open:

http://EC2-Public-IP

If this doesn't work, the problem is on EC2, not ALB.

---

**6. If Using Tomcat**

Check:

sudo systemctl status tomcat

or

ps -ef | grep tomcat

Verify Tomcat is listening:

sudo netstat -tulpn | grep 8080

If ALB Target Group uses port 8080, ensure Tomcat is running on 8080.

---

**Most Common Cause**

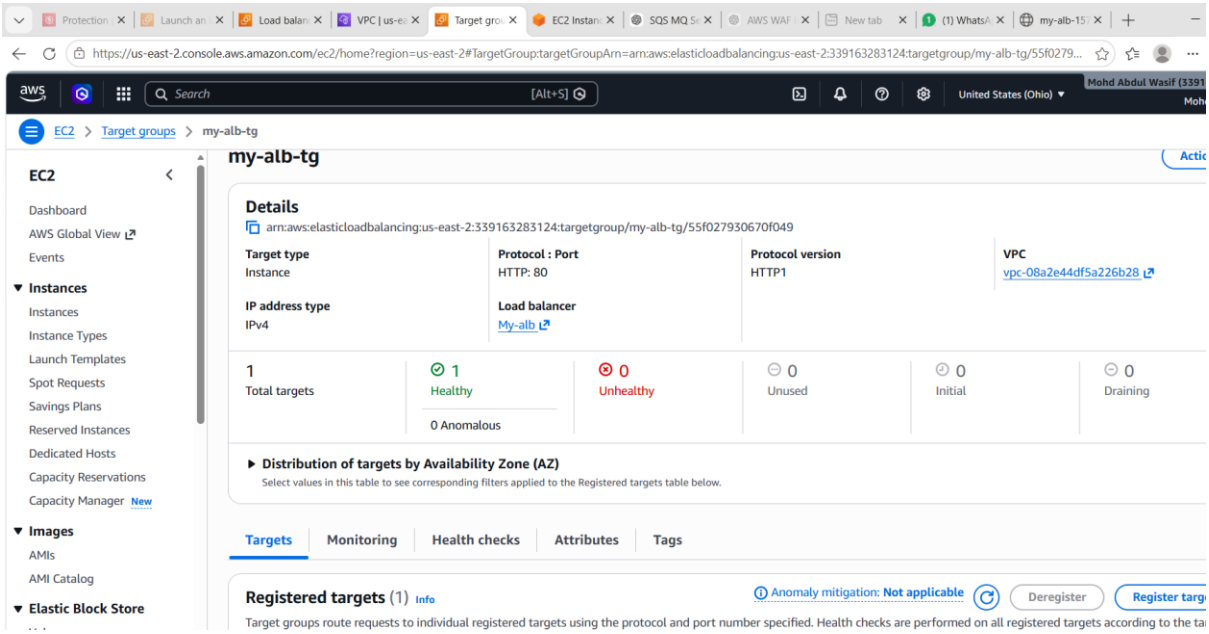
After creating ALB:

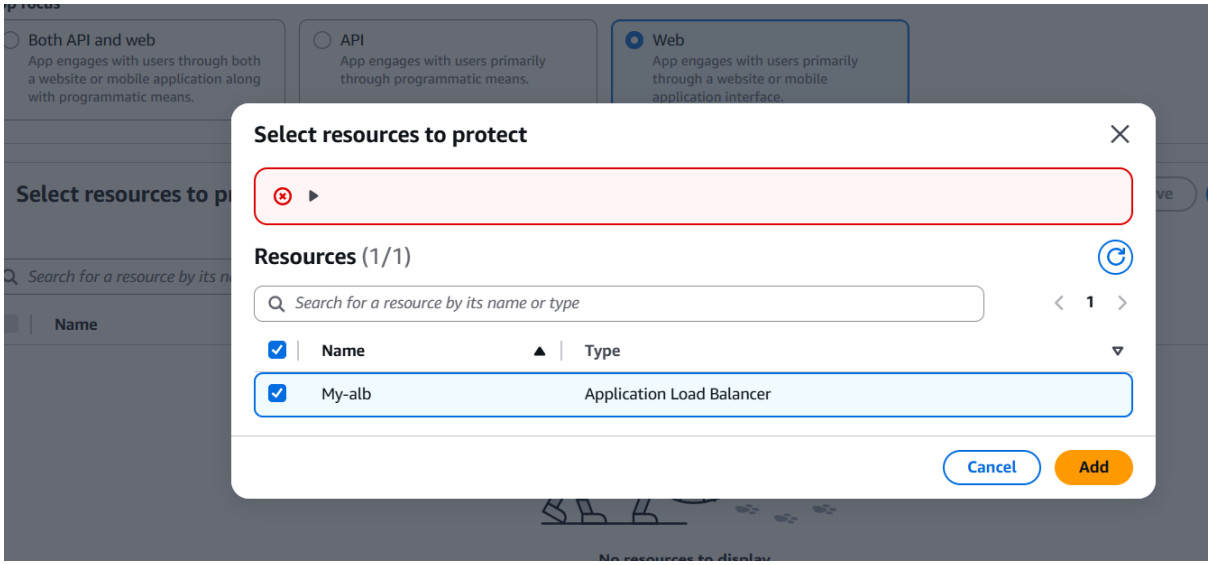
- Target Group port = 80
- Apache/Tomcat not running

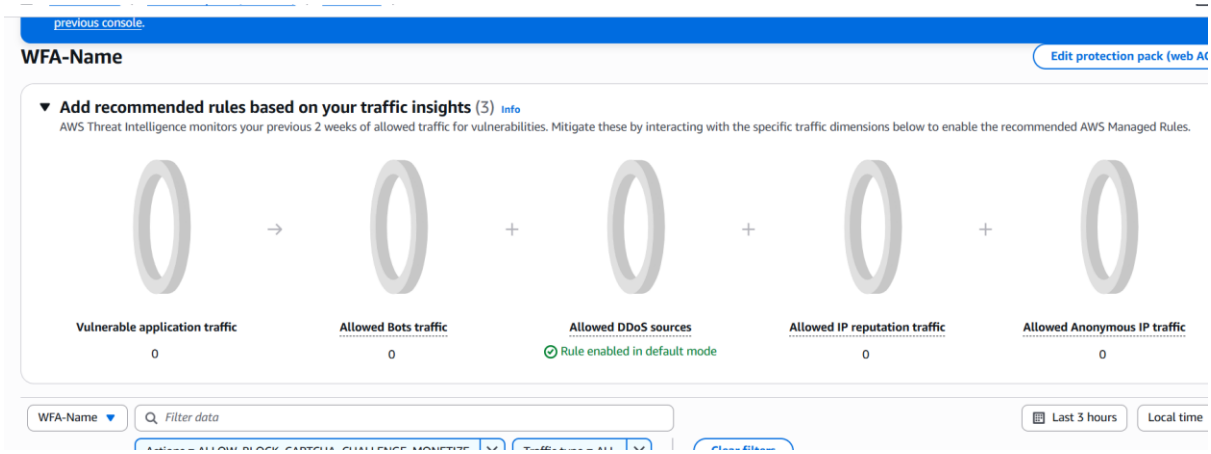
- Target marked **Unhealthy**
- ALB returns **503 Service Temporarily Unavailable**

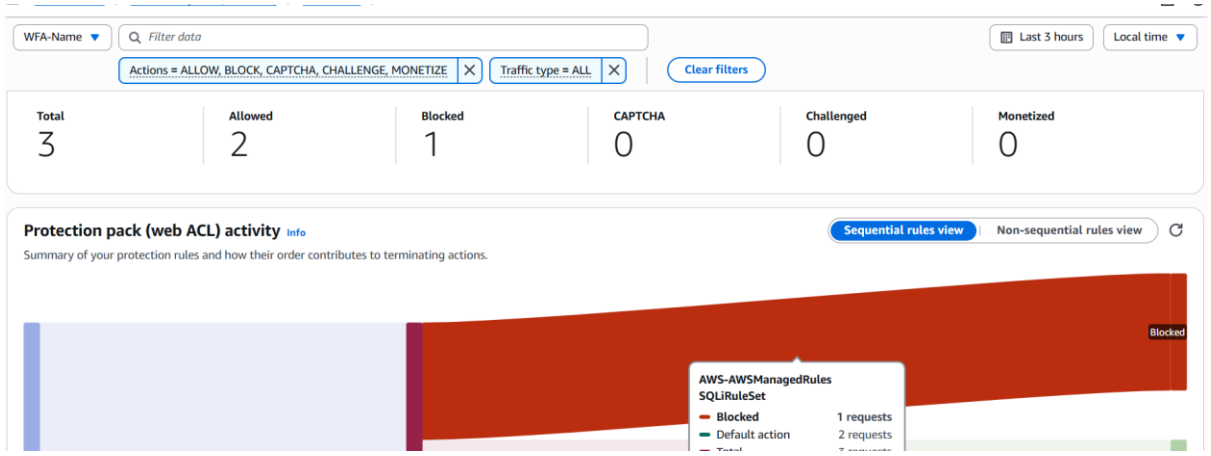
Tell me:

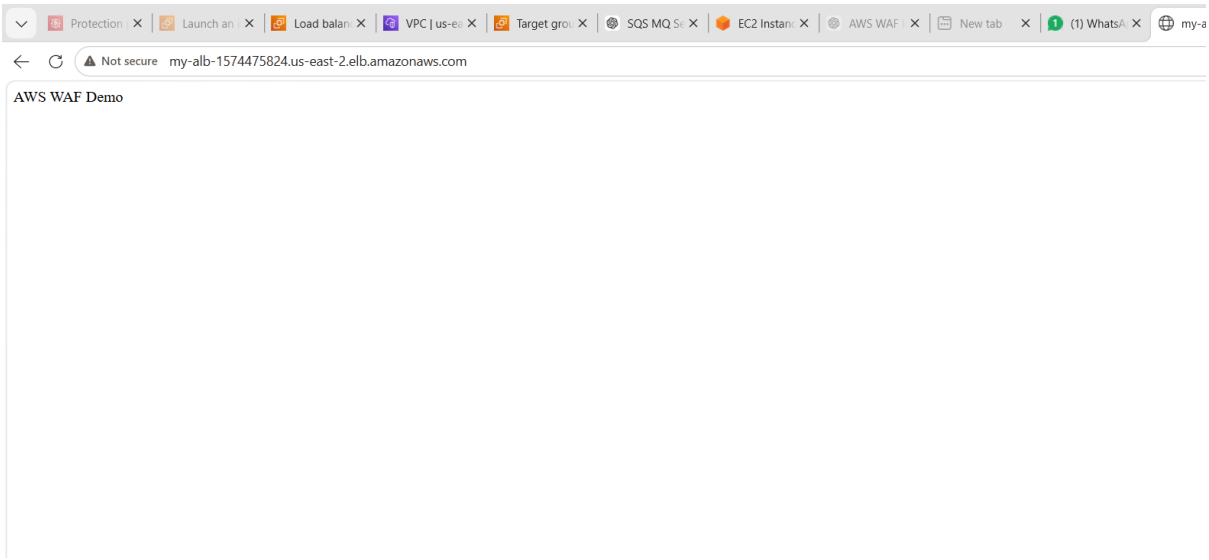
1. Are you using **Apache**, **Nginx**, or **Tomcat**?
2. What is the **Target Group port** (80 or 8080)?
3. What status do you see under **Target Group → Targets**?




Both API and web
App engages with users through both a website or mobile application along with programmatic means.
API
App engages with users primarily through programmatic means.
Web
App engages with users primarily through a website or mobile application interface.
Select resources to protect
Resources (1/1)
Search for a resource by its name or type
Name
Type
My-alb
Application Load Balancer
Cancel
Add


previous console.
WFA-Name
Edit protection pack (web AC
Add recommended rules based on your traffic insights (3) Info
AWS Threat Intelligence monitors your previous 2 weeks of allowed traffic for vulnerabilities. Mitigate these by interacting with the specific traffic dimensions below to enable the recommended AWS Managed Rules.
Vulnerable application traffic
0
Allowed Bots traffic
0
Allowed DDoS sources
Rule enabled in default mode
Allowed IP reputation traffic
0
Allowed Anonymous IP traffic
0
WFA-Name
Filter data
Last 3 hours
Local time


WFA-Name
Filter data
Last 3 hours
Local time
Actions = ALLOW, BLOCK, CAPTCHA, CHALLENGE, MONETIZE
Traffic type = ALL
Clear filters
Total
3
Allowed
2
Blocked
1
CAPTCHA
0
Challenged
0
Monetized
0
Protection pack (web ACL) activity Info
Summary of your protection rules and how their order contributes to terminating actions.
Sequential rules view
Non-sequential rules view
Blocked
AWS-AWSManagedRules
SQLiRuleSet
Blocked
1 requests
Default action
2 requests




**Step 4: Open correct ALB URL**

**Go to:**

**👉 EC2 → Load Balancers → WFA-INS**

**Copy DNS like:**

**http://wfa-ins-123456.us-east-2.elb.amazonaws.com**

**❌ Do NOT use:**

- **AWS WAF Demo ❌ (this is just page content)**
- **alb-dns ❌**

---

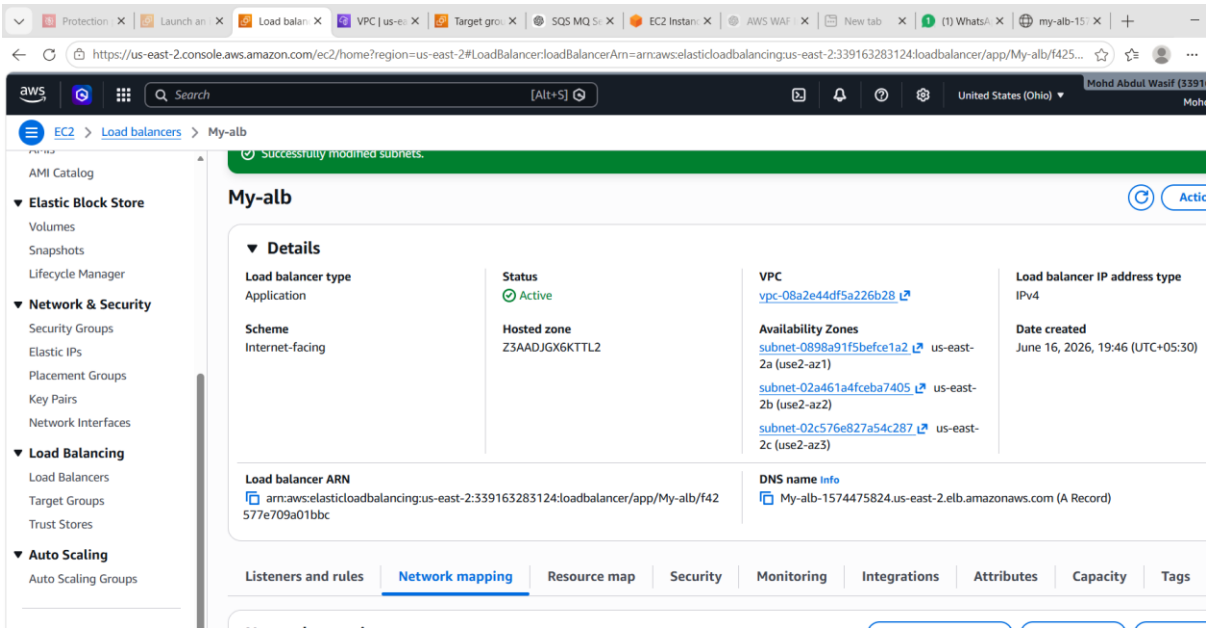
**🔥 Step 5: WAF check (after ALB works)**

**Go to:**

**👉 WAF → Web ACL → WFA-Name**

**Check:**

- **ALB is attached ✅**
- **Rules enabled ✅**




**Optional final test (for demo marks)**

**Try this in browser:**

**Normal request:**

http://my-alb-1574475824.us-east-2.elb.amazonaws.com

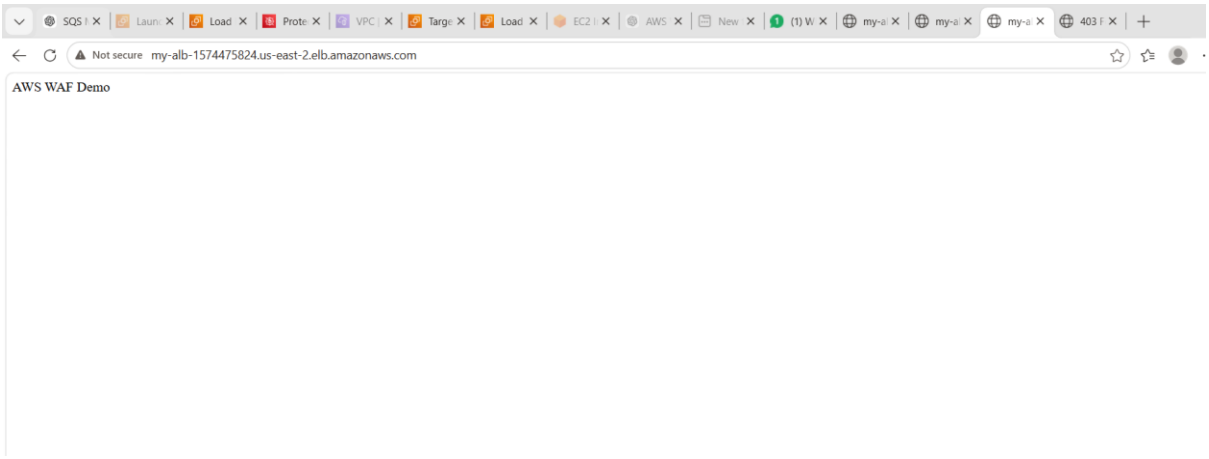
**Attack simulation:**

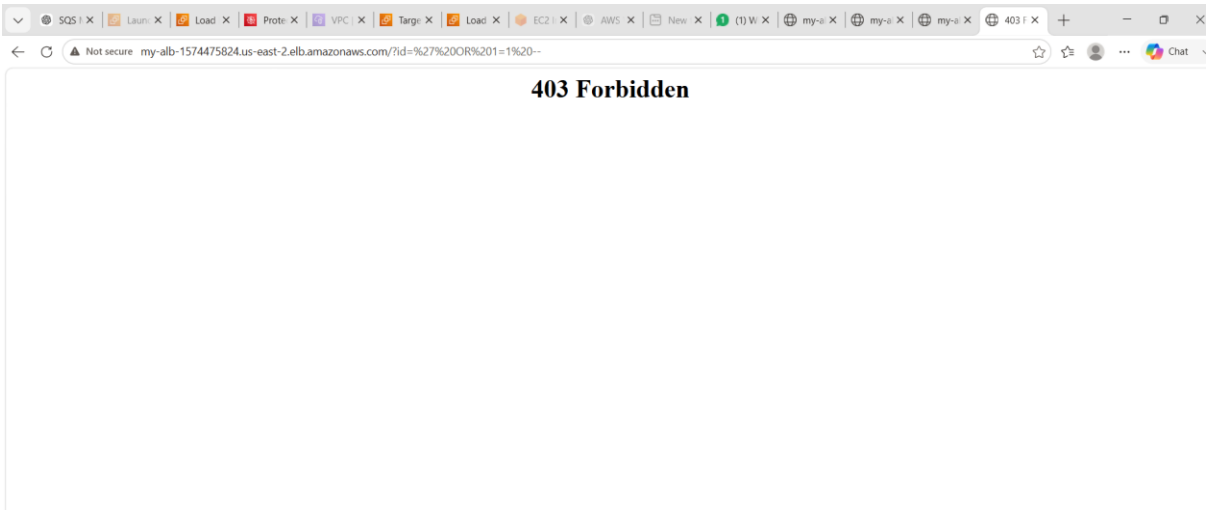
http://my-alb-1574475824.us-east-2.elb.amazonaws.com/?id=' OR 1=1 --

**If WAF is active:**

- **Normal → allowed** ✅
- **Malicious → blocked** ❌

---



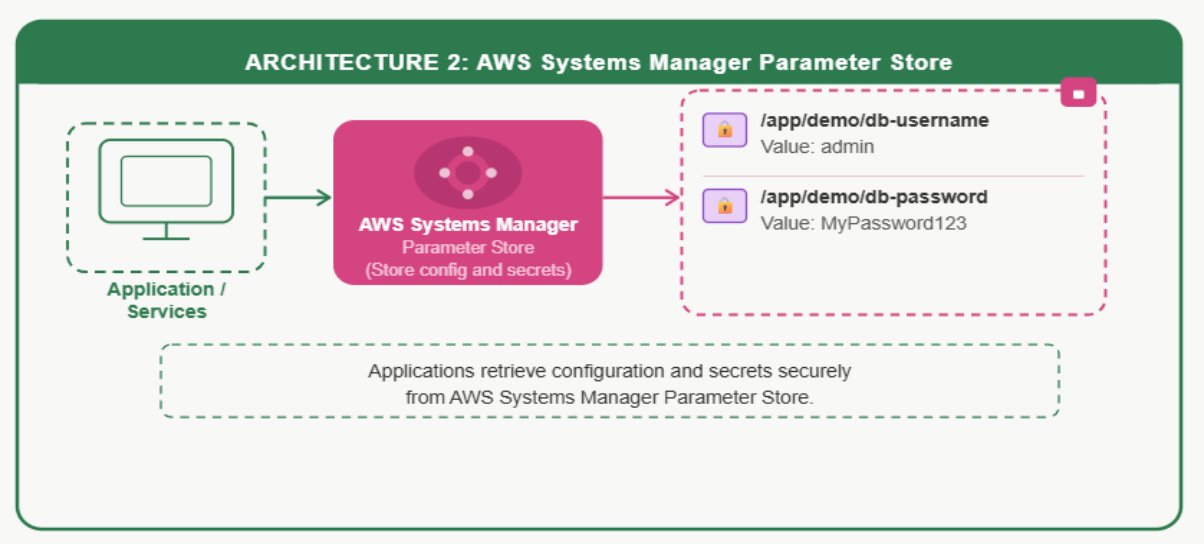




- # Final Status of your setup

✓ EC2 instance running
✓ Apache (httpd) working
✓ ALB DNS working
✓ Target group healthy
✓ WAF created and attached
✓ Page loading successfully

## 🎯 Conclusion (for your document)

Your AWS environment successfully demonstrates a secure web architecture using EC2, Application Load Balancer, and AWS WAF. The ALB routes traffic to a healthy EC2 instance, while WAF provides protection against malicious web requests.




## TASK 2: AWS SYSTEMS MANAGER PARAMETER STORE

### 🎯 Goal

Store and retrieve secure configuration values

---

### 🟢 STEP 1: Open Parameter Store

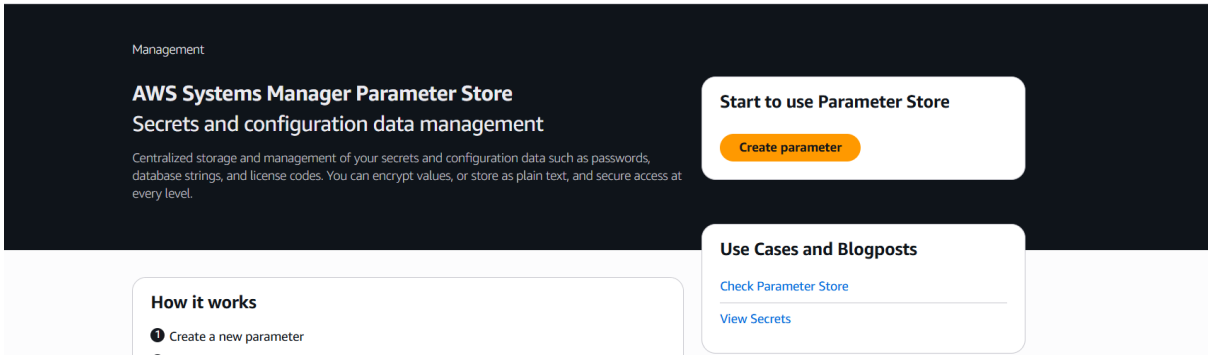
Go to AWS Systems Manager

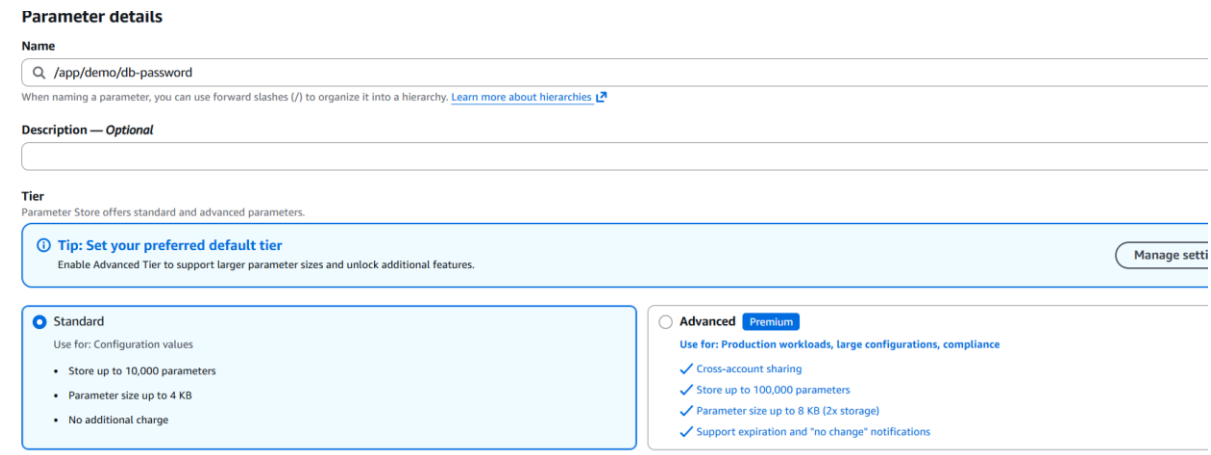
---

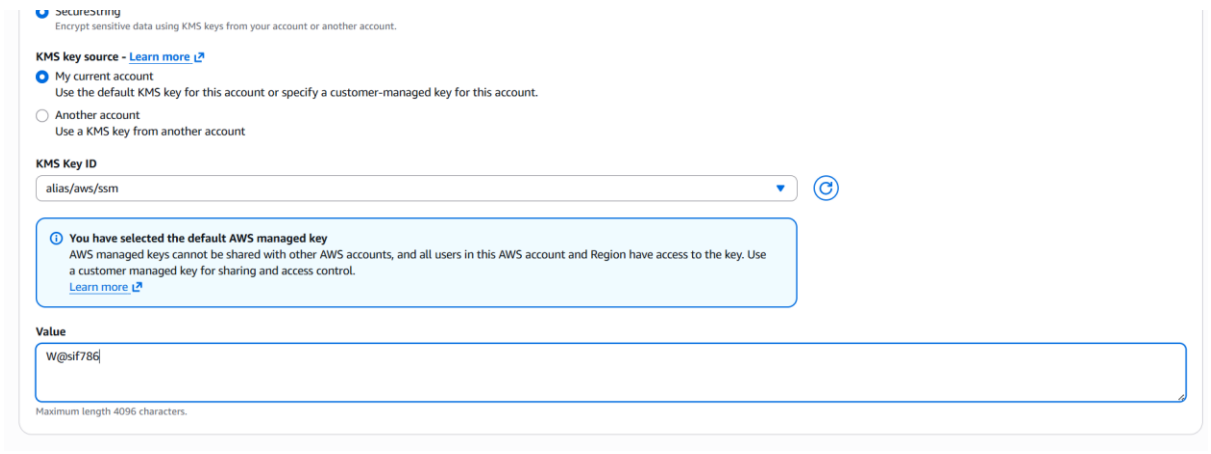
### 🟢 STEP 2: Create Parameters

**Parameter 1:**

- Name: /app/demo/db-password
- Type: String
- Value: MyPassword123

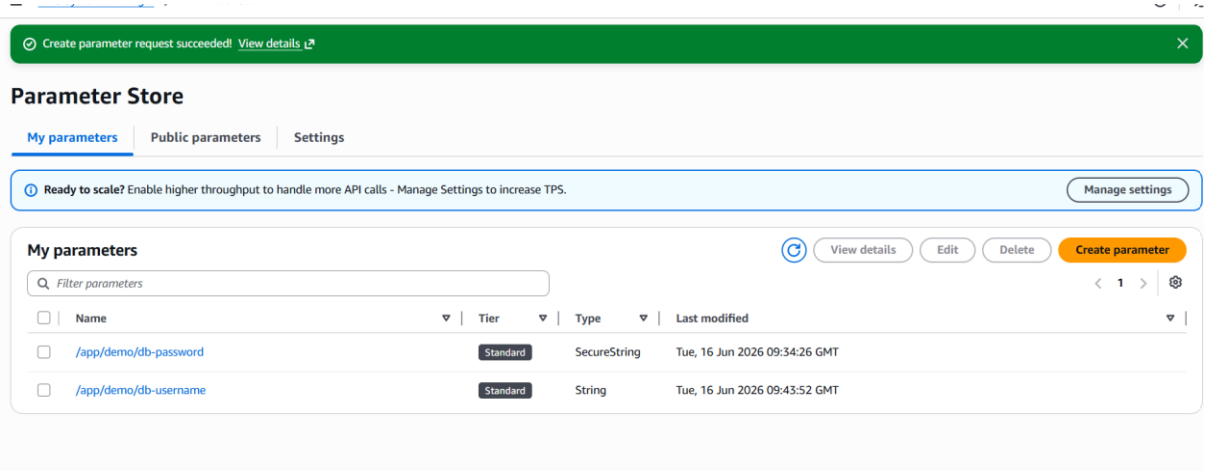

Management
AWS Systems Manager Parameter Store
Secrets and configuration data management
Centralized storage and management of your secrets and configuration data such as passwords, database strings, and license codes. You can encrypt values, or store as plain text, and secure access at every level.
Start to use Parameter Store
Create parameter
Use Cases and Blogposts
Check Parameter Store
View Secrets
How it works
1 Create a new parameter


Parameter details
Name
/app/demo/db-password
When naming a parameter, you can use forward slashes (/) to organize it into a hierarchy. Learn more about hierarchies
Description — Optional
Tier
Parameter Store offers standard and advanced parameters.
Tip: Set your preferred default tier
Enable Advanced Tier to support larger parameter sizes and unlock additional features.
Manage setti
Standard
Use for: Configuration values
Store up to 10,000 parameters
Parameter size up to 4 KB
No additional charge
Advanced
Premium
Use for: Production workloads, large configurations, compliance
Cross-account sharing
Store up to 100,000 parameters
Parameter size up to 8 KB (2x storage)
Support expiration and "no change" notifications
Type


Encrypt sensitive data using KMS keys from your account or another account.
KMS key source - Learn more
My current account
Use the default KMS key for this account or specify a customer-managed key for this account.
Another account
Use a KMS key from another account
KMS Key ID
alias/aws/ssm
You have selected the default AWS managed key
AWS managed keys cannot be shared with other AWS accounts, and all users in this AWS account and Region have access to the key. Use a customer managed key for sharing and access control.
Learn more
Value
W@sif786
Maximum length 4096 characters.

**Parameter 2:**

- Name: /app/demo/db-username
- Type: String
- Value: admin




**STEP 3: Verify in Console**

You should see both parameters listed

---

## 🟢 STEP 4: AWS CLI Setup Check

aws sts get-caller-identity

---

## 🟢 STEP 5: List Parameters

aws ssm describe-parameters --region us-east-1

---

## 🟢 STEP 6: Get Parameter (IMPORTANT)

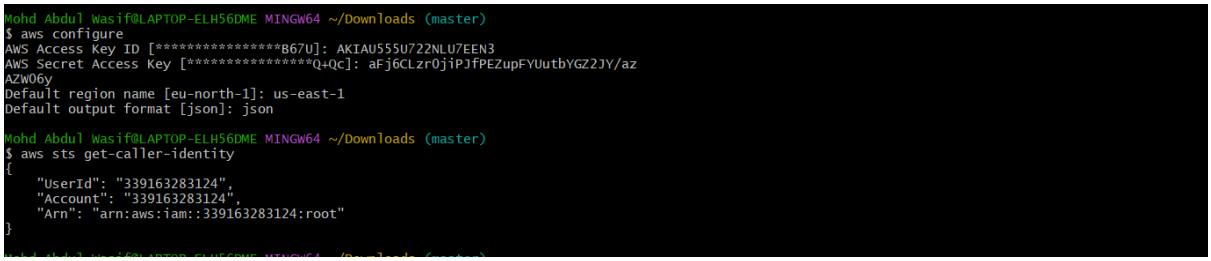
**Method 1:**

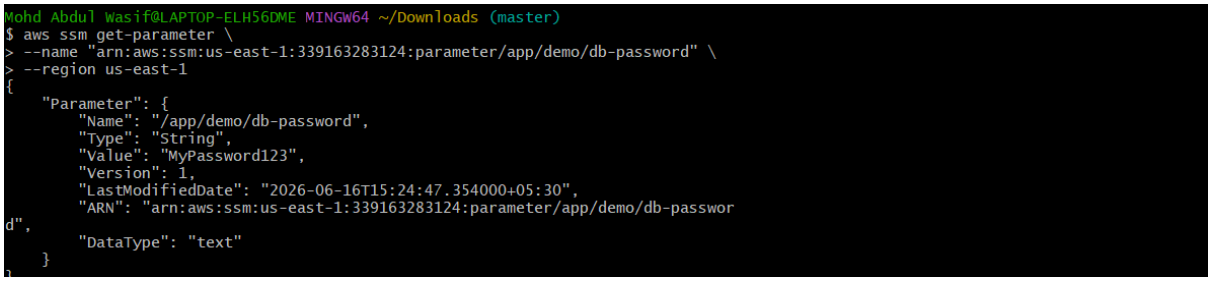
```
aws ssm get-parameter \
--name "/app/demo/db-password" \
--region us-east-1
```

**Method 2 (SAFE - YOU USED THIS):**

aws ssm get-parameter \
--name "arn:aws:ssm:us-east-1:ACCOUNT_ID:parameter/app/demo/db-password" \
--region us-east-1

---




**AWS Systems Manager Parameter Store**

- Created secure configuration parameters
- Stored database credentials
- Retrieved parameters using AWS CLI
- Used both name and ARN for retrieval

Application

↓

AWS Systems Manager Parameter Store